

```
#turtle.setheading(20) # Uncomment this line to draw the curve at an angle.
```

```
def hilbertCurveQuadrant(level, angle):  
    if level == 0:  
        # BASE CASE  
        return  
    else:  
        # RECURSIVE CASE  
        turtle.right(angle)  
        hilbertCurveQuadrant(level - 1, -angle)  
        turtle.forward(LINE_LENGTH)  
        turtle.left(angle)  
        hilbertCurveQuadrant(level - 1, angle)  
        turtle.forward(LINE_LENGTH)  
        hilbertCurveQuadrant(level - 1, angle)  
        turtle.left(angle)  
        turtle.forward(LINE_LENGTH)  
        hilbertCurveQuadrant(level - 1, -angle)  
        turtle.right(angle)  
        return
```

```
def hilbertCurve(startingPosition):  
    # Move to starting position.  
    turtle.penup()  
    turtle.goto(startingPosition)  
    turtle.pendown()  
    if DRAW_SOLID:  
        turtle.begin_fill()  
  
    hilbertCurveQuadrant(LEVELS, ANGLE) # Draw lower-left quadrant.  
    turtle.forward(LINE_LENGTH)  
  
    hilbertCurveQuadrant(LEVELS, ANGLE) # Draw lower-right quadrant.  
    turtle.left(ANGLE)  
    turtle.forward(LINE_LENGTH)  
    turtle.left(ANGLE)  
  
    hilbertCurveQuadrant(LEVELS, ANGLE) # Draw upper-right quadrant.  
    turtle.forward(LINE_LENGTH)  
  
    hilbertCurveQuadrant(LEVELS, ANGLE) # Draw upper-left quadrant.  
  
    turtle.left(ANGLE)  
    turtle.forward(LINE_LENGTH)  
    turtle.left(ANGLE)
```